
Retrieval- Augmented Generation (**RAG**)

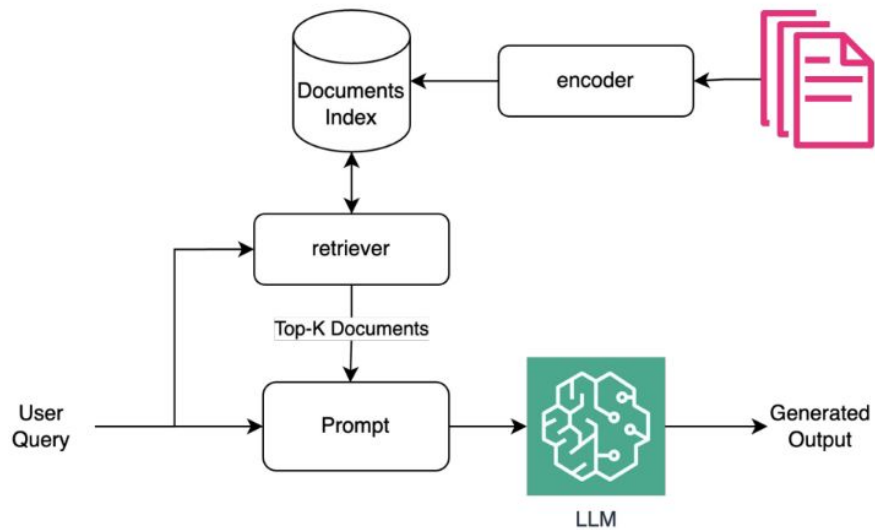
24.11.2025

Components

Index external documents

Retrieve documents relevant to query

Provide LLM with retrieved context



Implementation methodology

- Build naïve implementation
- Set up evaluation pipeline
- Iterate on the solution

Implementation methodology

- Build naïve implementation
- Set up evaluation pipeline
- Iterate on the solution

Split text

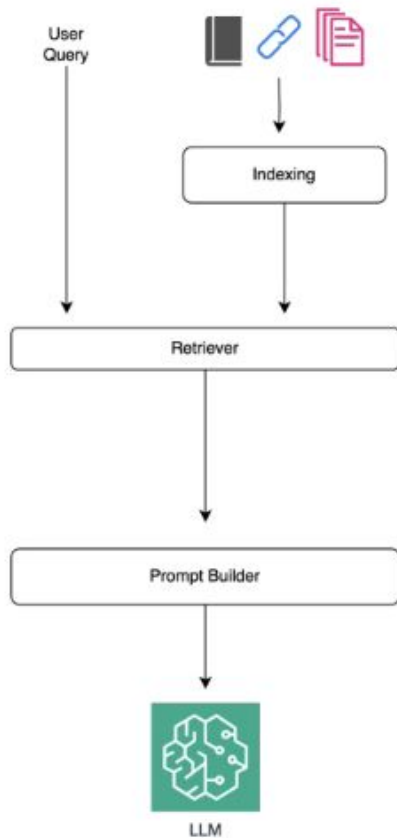
Embed text

Store vectors

Do nearest-neighbour search

Insert retrieved text into the prompt
of a local LLM

Implementation methodology



Simple example

Implementation methodology

- Build naïve implementation
- Set up evaluation pipeline
- Iterate on the solution



Retrieval Hit@K

if the correct answer appears in any of the top-k retrieved documents:

Hit@k = 1

Else:

0

Question: "Where is UCU located?"

Top-k retrieved docs include: "UCU is located in Lviv, Ukraine."

Hit@3 = 1

Implementation methodology

- E
- S
- I

What is
Recall@K?

Retrieval Hit@K

if the correct answer appears in any of the top-k retrieved documents:

Hit@k = 1

Else:

0

Question: "Where is UCU located?"

Top-k retrieved docs include: "UCU is located in Lviv, Ukraine."

Hit@3 = 1

Implementation methodology

- E
- S
- I

What is
Recall@K?

Retrieval Hit@K

if the correct answer appears in any of the top-k retrieved documents:

Hit@k = 1

Else:

0

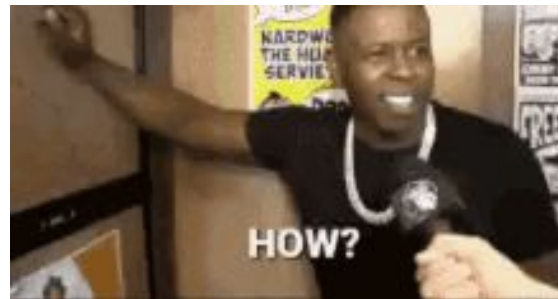
$$\text{Recall@K} = \frac{\# \text{Relevant in Top K}}{\# \text{All Relevant Documents}}$$

Hit@3 = 1

Implementation methodology

- Build naïve implementation
- Set up evaluation pipeline
- Iterate on the solution

But How to improve RAG?



Improving RAG

By better
Chunking

By optimizing
Query

By adding
Routing

By better
Indexing

Chunking Strategies

Fixed-Size

Best Fit: Relatively uniform documents with consistent formatting, such as simple **logs** or straightforward text.

Advantages

- Straightforward and easy to implement.
- Uniform chunk size simplifies batch operations.
- Works decently for content that doesn't heavily rely on semantic context

```
text_splitter = CharacterTextSplitter(  
    separator="\n\n",          # split on paragraph  
    chunk_size=chunk_size,  
    chunk_overlap=chunk_overlap,  
    length_function=len,  
)
```

Drawbacks

- May cut off sentences or paragraphs.
- Ignores natural semantic breaks.
- Relevant information can end up spread across chunks

Chunking Strategies

Fixed-Size

Best Fit: Relatively uniform documents with consistent formatting, such as simple **logs** or straightforward text.

Advantages

- Straightforward and easy to implement
- Uniform chunk size simplifies batch operations
- Works decently for content that doesn't rely on semantic context

1. Split document by paragraphs.
2. Start building a chunk by concatenating whole paragraphs until:

Check if total length $\geq$ chunk_size

If True:

move last paragraph to the next chunk.

3. Repeat.
4. Add overlap.

paragraph

Chunking Strategies

Fixed-Size

Best Fit: Relatively uniform documents with consistent formatting, such as simple **logs** or straightforward text.

Advantages

- Straightforward and easy to implement
- Uniform chunk size simplifies batch operations
- Works decently for content that doesn't rely on semantic context

1. Split document by paragraphs.
2. Start building a chunk by concatenating whole paragraphs until:

Check if total length $\geq$ chunk_size

If True:

move last paragraph to the next chunk.

3. Repeat
4. Add overlap.

Overlap?

Chunking Strategies

Fixed-Size

Best Fit: Relatively uniform documents with consistent formatting, such as simple **logs** or straightforward text.

Advantages

- Straightforward and easy to implement
- Uniform chunk size simplifies batch operations
- Works decently for content that doesn't rely on semantic context

1. Split document by paragraphs.
2. Start building a chunk by concatenating whole paragraphs until:

Check if total length $\geq$ chunk_size

If True:

move last paragraph to the next chunk.

3. Repeat
4. Add

Overlap?

When a portion of the end of one chunk is duplicated at the beginning of the next chunk

paragraph

Chunking Strategies

Semantic

Best Fit: Well-structured, narrative, or academic documents where continuity is crucial.

sentences, paragraphs, or sections

Advantages

- Preserves the flow of ideas.
- Keeps related concepts together, boosting retrieval accuracy.
- Helpful for documents like articles or academic papers that have clear sections.

```
text_splitter = RecursiveCharacterTextSplitter(  
    separators=["\n\n", "\n", ". ", " ", ""],  
    chunk_size=chunk_size,  
    chunk_overlap=chunk_overlap,  
    length_function=len,  
)
```

Drawbacks

- More complex to implement.
- Chunk sizes may vary.
- Slightly higher computational requirements.

Chunking Strategies

Recursive

Best Fit: Technical documents with a clear structure, especially code repositories or structured reports.

```
code_splitter = RecursiveCharacterTextSplitter.from_language(  
    language=Language.GO,  
    chunk_size=chunk_size,  
    chunk_overlap=chunk_overlap,  
)
```

Advantages

- Creates more context-aware splits than simple fixed-size approaches.
- Especially powerful for structured text or code, where block-based splitting is crucial.

Drawbacks

- More complicated to configure.
- Requires domain-specific separators for best results (like `def` or `class` in Python).

Chunking Strategies

Adaptive

Best Fit: Mixed-content documents with varying degrees of complexity, such as technical handbooks containing both simple descriptions and advanced in-depth analyses.

Advantages

- Dynamically allocates resources to complex sections.
- Reduces unnecessary token usage on simpler parts.
- Can provide a more nuanced approach to chunking.

```
if self.complexity_measure in ("lexical_density", "combined"):
    words = re.findall(pattern: r"\b\w+\b", text.lower())
    if not words:
        lex_density = 0.0
    else:
        unique_words = set(words)
        lex_density = len(unique_words) / len(words)

    # Normalize between 0 and 1, max lex_density of 0.8
    lex_density = min(1.0, lex_density / 0.8)
else:
    lex_density = 0.0
```

Drawbacks

- Requires a complexity function or metric. (to eval. the text)
- More difficult to debug or tune.
- Demands more computation up front.

Chunking Strategies

LLM-driven

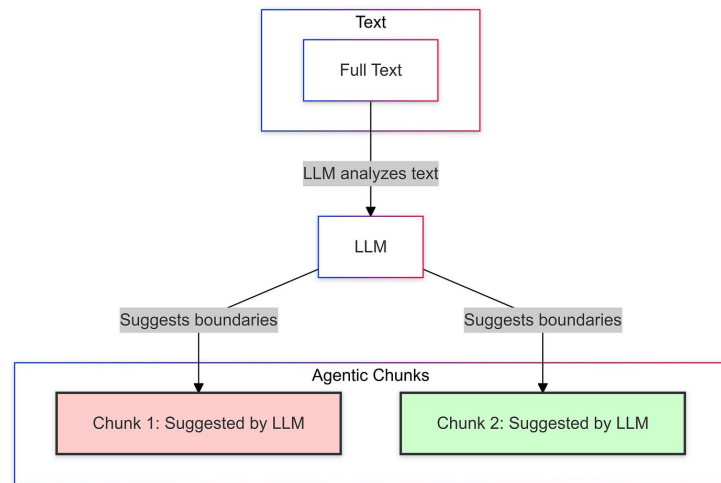
Best Fit: Projects with generous compute budgets and a critical need for accurate, context-driven chunk segmentation.

Advantages

- Highly adaptive, capturing semantic nuance.
- Can significantly enhance retrieval accuracy.
- Especially useful for complex, multi-topic documents.

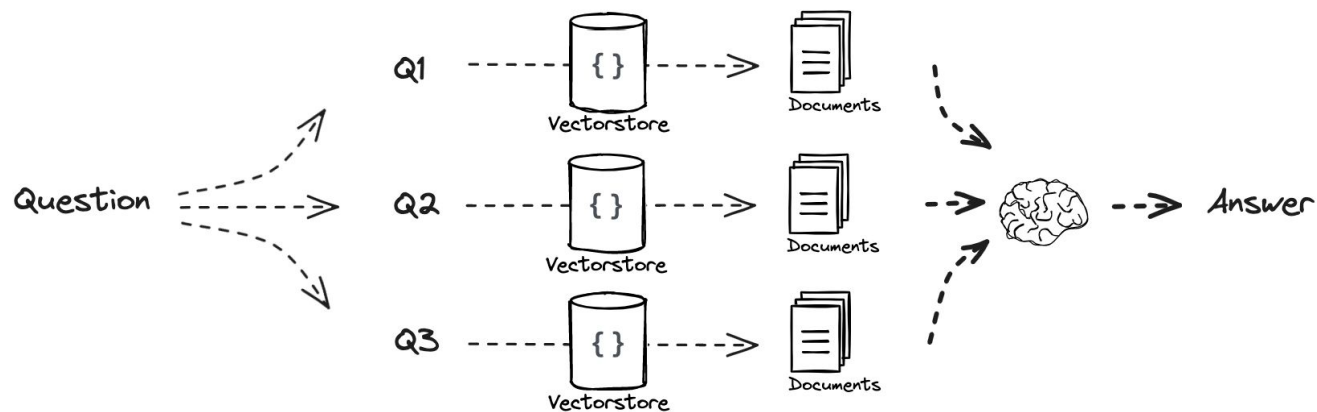
Drawbacks

- Relies on the performance of the underlying LLM.
- Potentially expensive in terms of compute and API calls.
- Harder to standardize or replicate consistently.



Optimizing Query

Multi query



Multi query

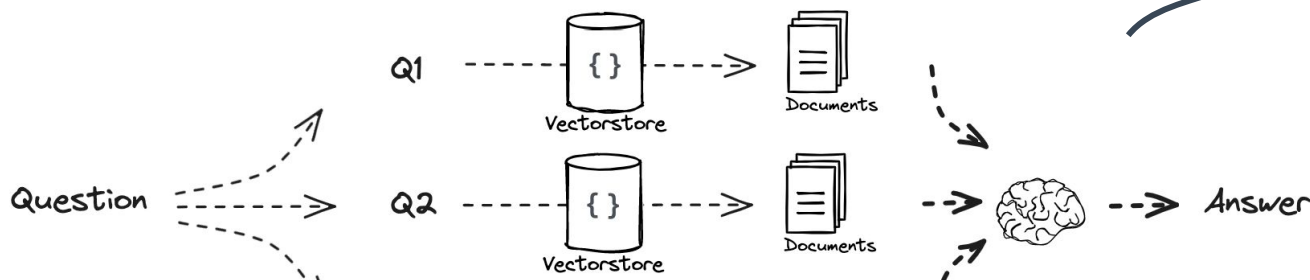
Optimizing Query

```
system_msg = (  
    "You rewrite user questions for retrieval in a RAG system. "  
    "Return exactly N alternative queries that might retrieve relevant documents. "  
    "Output only a JSON array of strings."  
)  
  
user_msg = (  
    f"Question: {question}\n"  
    f"N = {num_rewrites}\n"  
    "Return something like:\n"  
    '["rewrite 1", "rewrite 2", "rewrite 3"]'  
)  
  
resp = client.chat.completions.create(  
    model="gpt-4.1-mini",  
    messages=[  
        {"role": "system", "content": system_msg},  
        {"role": "user", "content": user_msg},  
    ],  
    temperature=0.3,  
)
```

Optimizing Query

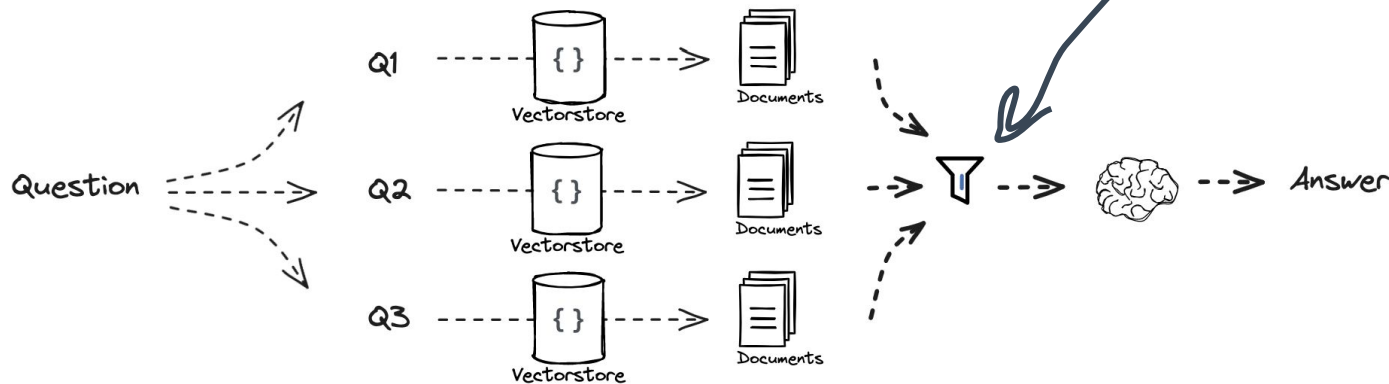
RAG-Fusion

<https://github.com/Raudaschl/rag-fusion>



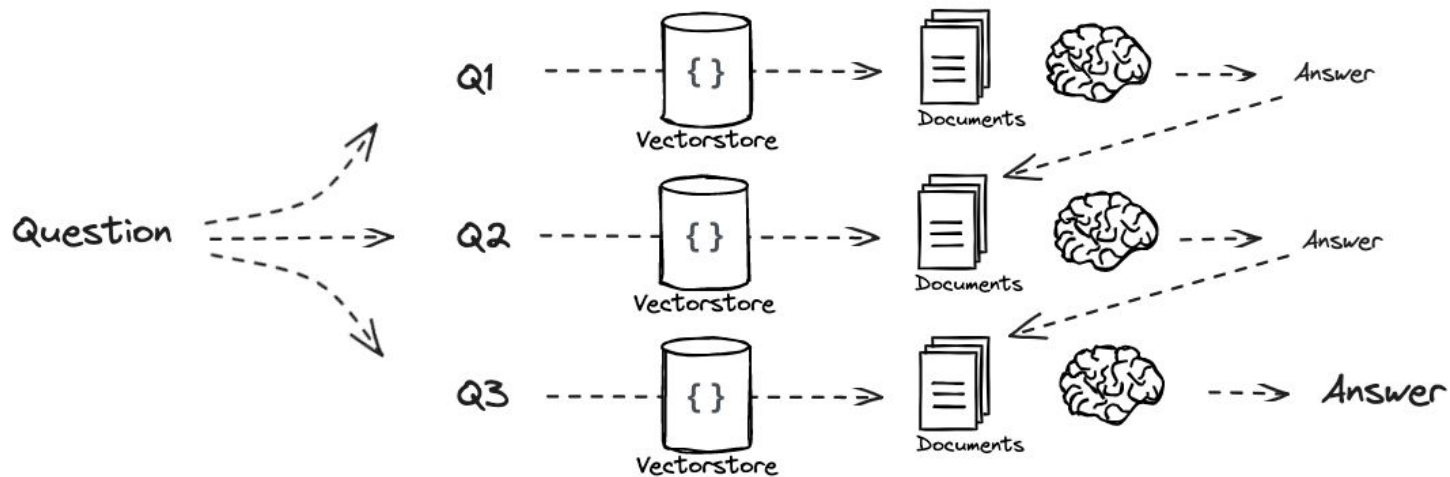
Reciprocal Rank Fusion:

Applies the Reciprocal Rank Fusion algorithm to re-rank the documents based on their relevance across multiple queries.



Optimizing Query

Decomposition



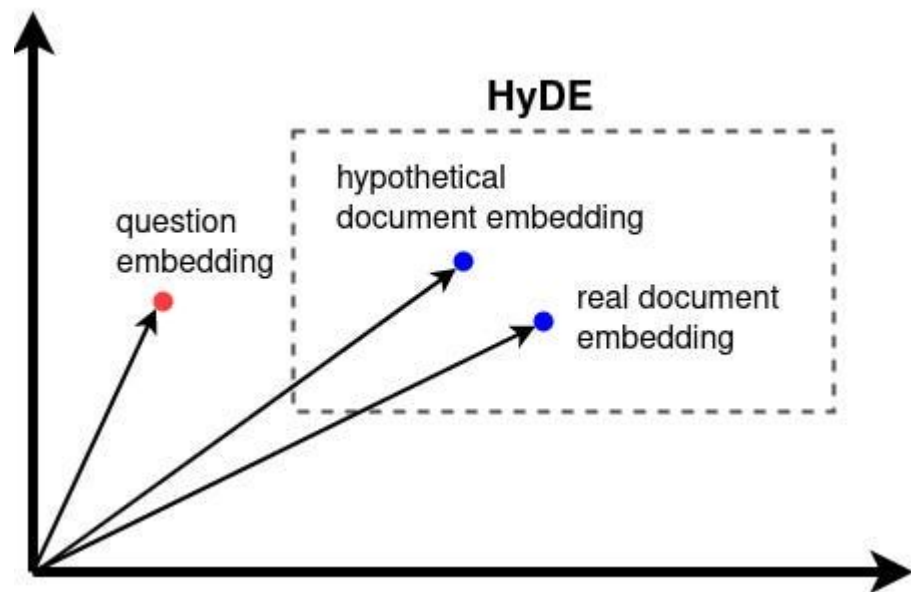
Optimizing Query

Decomposition

```
system_msg = (  
    "Break the user question into smaller sub-questions required to answer it. "  
    "Return exactly N sub-questions in JSON array form."  
)  
user_msg = (  
    f"N={n}\n"  
    f"Question: {question}\n"  
    'Return format: ["subQ1", "subQ2", ...]'  
)  
  
resp = client.chat.completions.create(  
    model="gpt-4.1-mini",  
    messages=[  
        {"role": "system", "content": system_msg},  
        {"role": "user", "content": user_msg}  
    ],  
    temperature=0.3  
)
```

Optimizing Query

HyDE



Optimizing Query

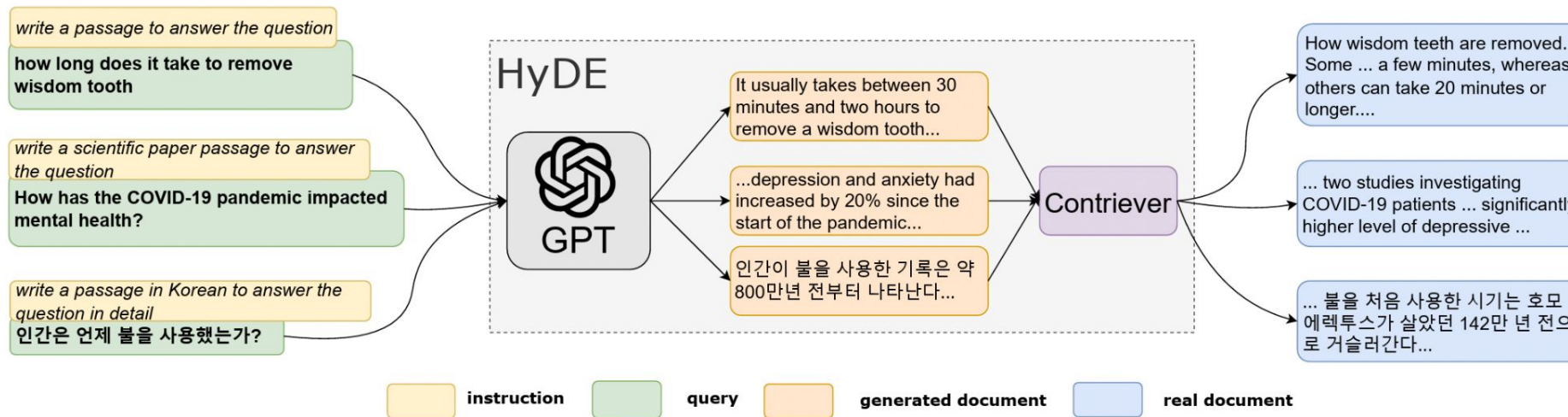
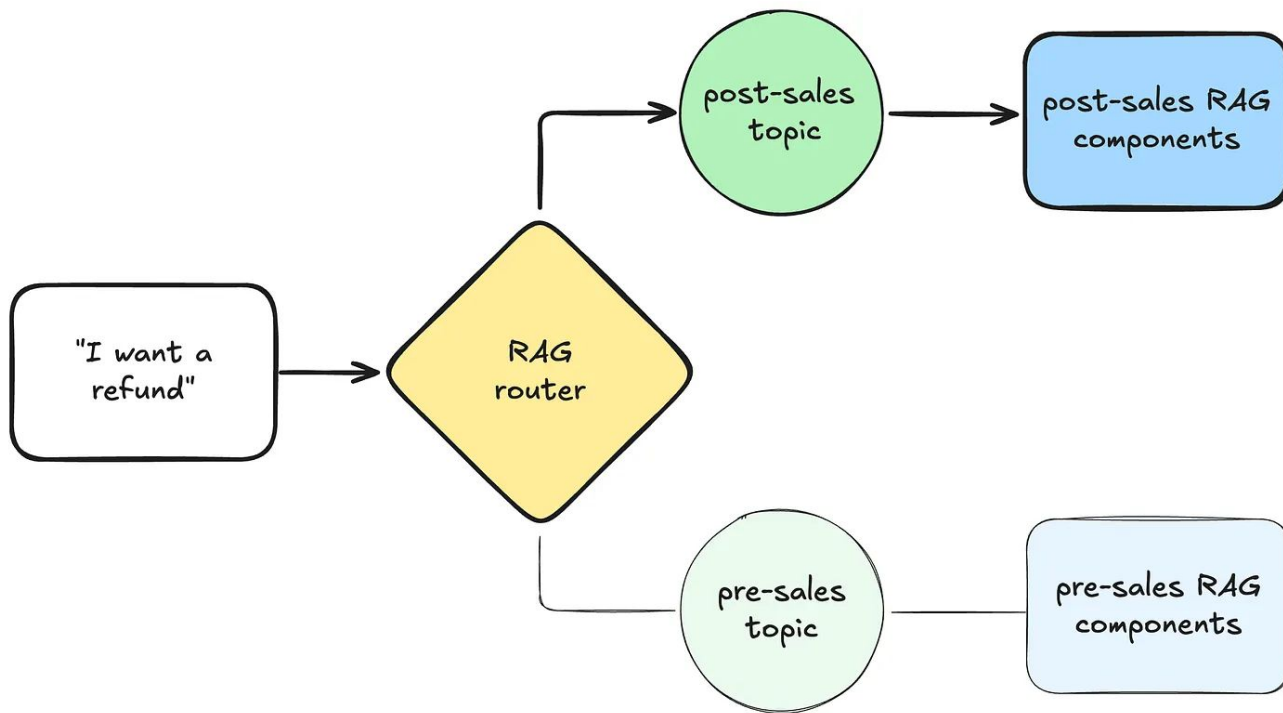


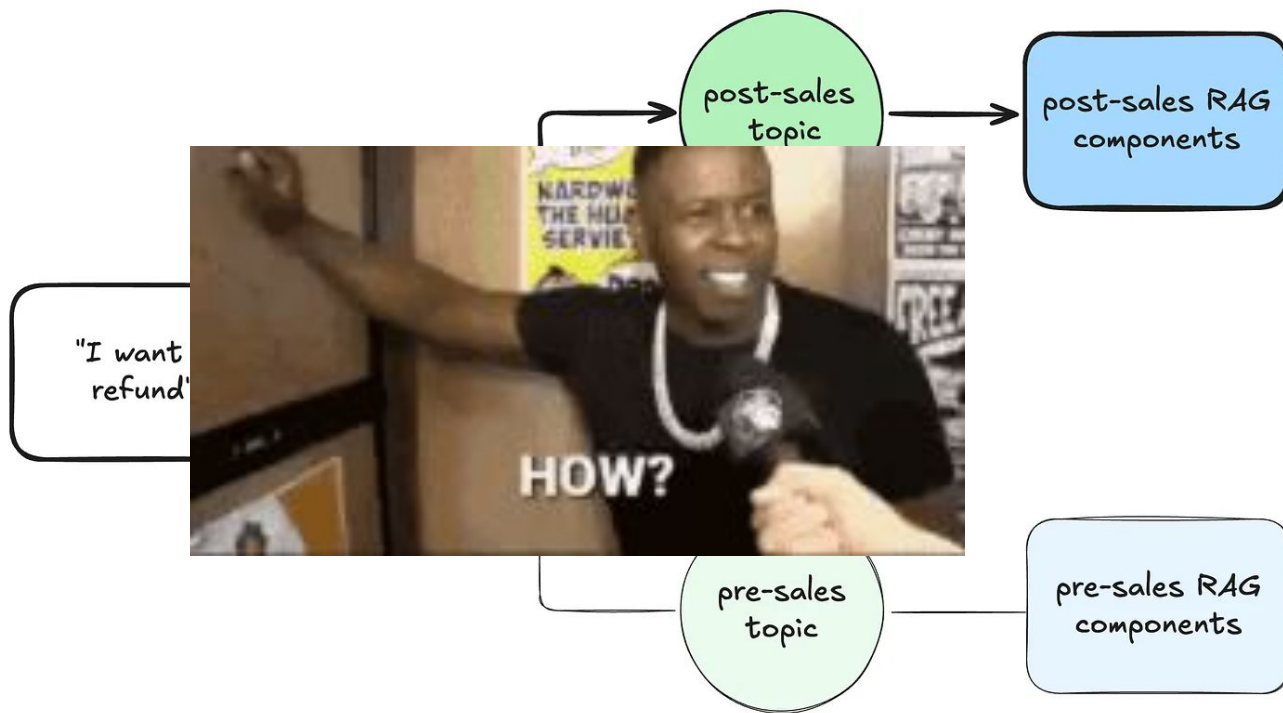
Figure 1: An illustration of the HyDE model. Documents snippets are shown. HyDE serves all types of queries without changing the underlying GPT-3 and Contriever/mContriever models.

<https://arxiv.org/abs/2212.10496>

Routing



Routing



Routing

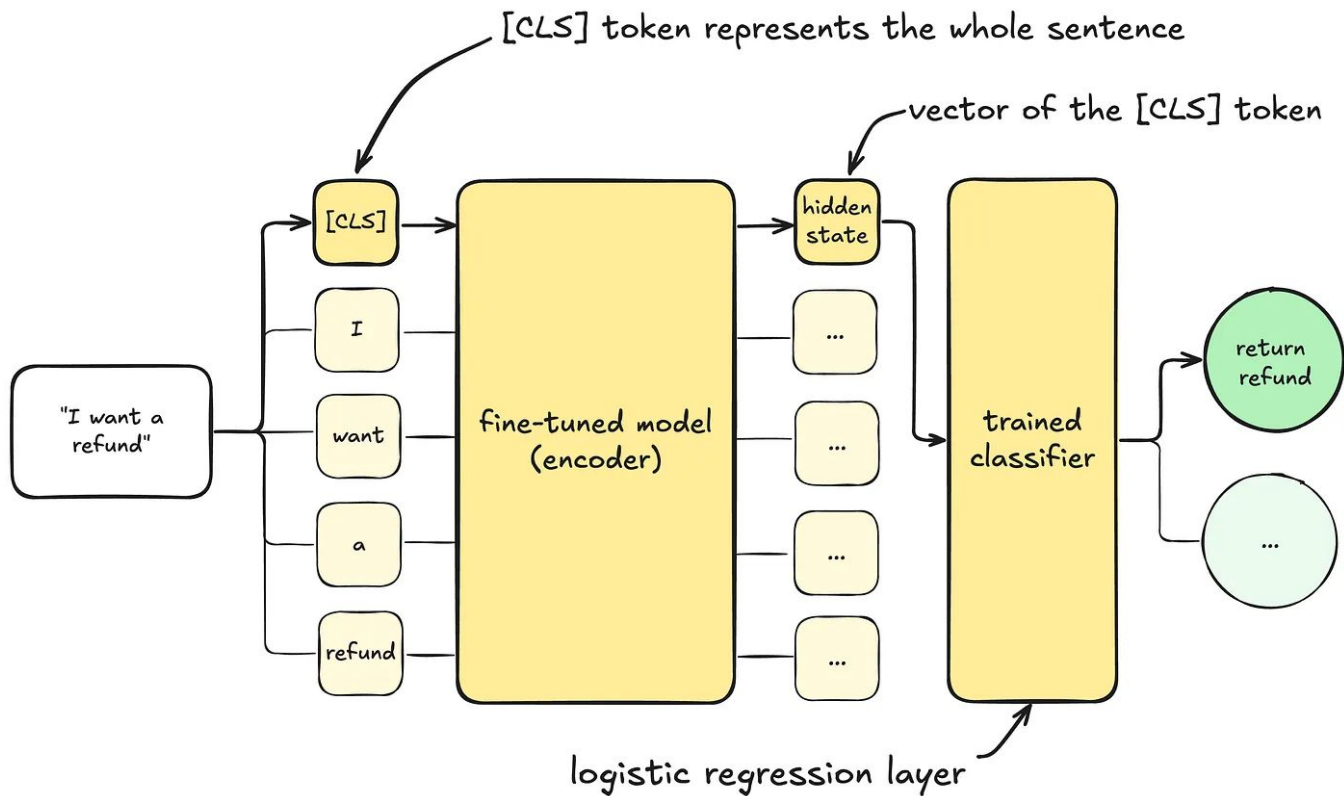
LLM
based

Rule
based

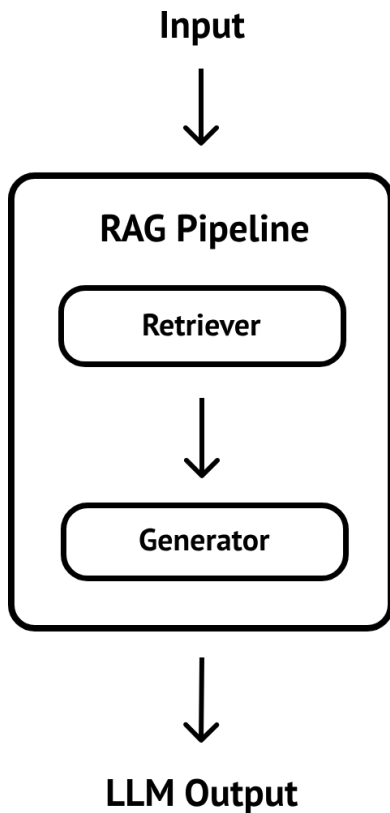
Multi-
retriever

Tune your
own!

Routing



Evaluate



Colab [Example DeepEval](#)



Retriever Metrics:

- Contextual Recall
- Contextual Precision
- Contextual Relevancy

Generator Metrics:

- Answer Relevancy
- Faithfulness

<https://deepeval.com/guides/guides-rag-evaluation>

